

An End-to-End MLOps Pipeline for Fake News Detection: Evaluating Reproducibility, Deployment Efficiency, Runtime Performance, and Automated Drift Detection

Tabidah Usmani*, Amna Javaid†, Sara Zahid‡

Department of Data Science

National University of Computer and Emerging Sciences

Islamabad, Pakistan

*i222070@nu.edu.pk †i222025@nu.edu.pk ‡i221861@nu.edu.pk

Abstract—The deployment of machine learning models in production environments presents significant challenges including poor reproducibility, inconsistent environments, and absence of automated monitoring. This paper presents an end-to-end Machine Learning Operations (MLOps) pipeline for fake news detection and empirically evaluates its impact on reproducibility, deployment efficiency, runtime performance, and automated drift detection compared to traditional manual workflows. Using the WELFake dataset of 72,134 news articles, we train and compare three classifiers—Logistic Regression, Random Forest, and LinearSVC—with LinearSVC achieving the highest F1-score using an improved TF-IDF feature extraction pipeline with sublinear term-frequency scaling. The pipeline integrates MLflow for experiment tracking, Docker for containerization of seven services, GitHub Actions for CI/CD automation, and Prometheus with Grafana for real-time monitoring. A novel sliding-window prediction drift detector monitors the FAKE/REAL label distribution over the last 50 predictions and automatically triggers model retraining via Alertmanager and a dedicated retrain webhook service when the ratio exceeds a 70% threshold. Experimental results demonstrate 100% reproducibility across three independent training runs, reduction in deployment time from approximately 15 minutes to 78 seconds via CI/CD automation, stable inference latency averaging 123.06 ms, and successful automated drift alerting confirmed through Prometheus metrics and Grafana dashboards. Our findings validate that structured MLOps practices substantially improve the reliability, efficiency, and observability of NLP-based classification systems in production settings.

Index Terms—MLOps, fake news detection, machine learning pipeline, Docker, MLflow, CI/CD, Prometheus, Grafana, Alertmanager, drift detection, reproducibility, NLP, deployment, LinearSVC

I. INTRODUCTION

The proliferation of misinformation on digital platforms has emerged as a critical societal challenge, influencing public opinion, political discourse, and institutional trust [1]. Automated fake news detection using machine learning has attracted considerable research attention; however, transition-

ing these models from experimental notebooks to reliable production systems remains a persistent challenge [5].

Traditional model deployment practices suffer from three fundamental problems. First, experiments are difficult to reproduce due to undocumented hyperparameters, inconsistent environments, and absent version control for models and data. Second, manual deployment processes are slow, error-prone, and non-standardized. Third, deployed models operate without visibility into runtime behavior, making it impossible to detect performance degradation or data drift without manual inspection [6].

Machine Learning Operations (MLOps) addresses these challenges by applying DevOps principles to the machine learning lifecycle, enabling continuous integration, continuous delivery, and continuous monitoring of ML systems [10]. Despite growing industrial adoption, empirical validation of MLOps benefits in the context of NLP classification tasks—particularly fake news detection—remains limited in the existing literature.

This paper extends our initial pipeline design with three additional contributions: (1) a third classifier (LinearSVC) with improved TF-IDF feature extraction, (2) a production-grade automated drift detection system using sliding-window prediction monitoring, and (3) a fully automated retraining loop integrating Prometheus alerting, Alertmanager routing, and a dedicated retrain webhook service that triggers GitHub Actions workflows on drift detection.

This paper makes the following contributions:

- Design and implementation of a complete MLOps pipeline for fake news detection integrating seven production-grade services: MLflow, Docker, GitHub Actions, Prometheus, Grafana, Alertmanager, and a retrain webhook service.
- Empirical evaluation of three classifiers (Logistic Regression, Random Forest, LinearSVC) with improved TF-IDF

featuring sublinear term-frequency scaling, auto-selecting the best model by F1-score.

- Empirical evaluation of reproducibility demonstrating 100% metric consistency across three independent training runs with controlled random state.
- Quantitative measurement of deployment efficiency, showing CI/CD automation reduces deployment time by over 90% compared to manual workflows.
- A sliding-window prediction drift detector that monitors label distribution over the last 50 predictions and automatically triggers a full retraining pipeline when the FAKE or REAL ratio exceeds 70%.
- Confusion matrix analysis and per-class error breakdown, confirming the pipeline’s suitability for production deployment where false negatives carry greater societal cost.

The remainder of this paper is organized as follows. Section II reviews related work. Section III defines research questions and hypotheses. Section IV describes methodology and system architecture. Section V presents experimental results. Section VI discusses findings. Section VII concludes the paper.

II. RELATED WORK

A. Fake News Detection

Mouratidis et al. [1] present a comprehensive ML framework for fake news detection, systematically comparing traditional classifiers (Naïve Bayes, SVM, Random Forest) with deep learning models (CNN, LSTM, BERT). Their findings demonstrate that BERT-based models achieve superior performance but at significantly higher computational cost, while Random Forest provides competitive accuracy with substantially lower inference overhead. This supports our choice of classical ML models for the production pipeline.

Hoy and Cosma [2] demonstrate that existing NLP approaches perform well under cross-validation but struggle to generalise across datasets due to publisher-level biases in training labels. This finding directly motivates our automated retraining pipeline: as misinformation patterns evolve, the model must adapt continuously—a core capability of the drift-triggered retraining system we implement.

Baron and Alsallami [3] evaluate CNN, BERT, and GPT variants for fake news classification, reporting that fine-tuned GPT-4 achieves 98.6% accuracy but requires substantially more infrastructure. Their work confirms that simpler models are preferable for production deployments where latency and resource efficiency are constraints, a trade-off our LinearSVC selection reflects.

A comprehensive review by Rampurkar et al. [4] identifies reproducibility and deployment consistency as key open challenges, which our work directly addresses through MLflow tracking and containerized execution.

B. MLOps Frameworks and Pipelines

Kreuzberger et al. [5] propose an MLOps framework using GitHub Actions, Jenkins, Docker, MLflow, Prometheus, and Grafana, demonstrating improvements in model accuracy exceeding 7% and significant reductions in deployment latency

through automated retraining. Their stack closely mirrors our implementation; however, their evaluation does not target NLP classification tasks and does not implement prediction drift detection at the API level.

John et al. [6] present a comprehensive MLOps pipeline for diabetes prediction using FastAPI, Docker, and Kubernetes, achieving 98.3% reproducibility consistency across heterogeneous computing environments and reducing deployment friction by 73% compared to traditional workflows. Their reproducibility methodology directly informs our experimental design.

A meta-synthesis of MLOps practices [7] identifies containerization, CI/CD pipelines, and experiment tracking as the three most impactful components of production ML systems. Houerbi et al. [8] further demonstrate that production-grade CI/CD pipelines for ML projects remain rare in practice, with most implementations focusing on reporting rather than automated retraining and deployment—a gap our retrain webhook directly addresses.

Bagai et al. [9] explore CI/CD strategies for ML on AWS platforms, recommending Prometheus, Grafana, and AWS CloudWatch for model monitoring and drift detection. We implement and validate the Prometheus-Grafana component of this recommendation and extend it with Alertmanager-based automated retraining triggers.

III. RESEARCH QUESTIONS AND HYPOTHESES

A. Primary Research Question

How does an automated MLOps pipeline improve reproducibility, deployment efficiency, runtime performance, and drift resilience of fake news detection systems compared to traditional manual workflows?

B. Sub-Research Questions

RQ1: How does MLflow experiment tracking improve model reproducibility across multiple independent training runs?

RQ2: What is the impact of Docker containerization on deployment consistency and inference latency?

RQ3: How does CI/CD automation reduce deployment time compared to manual processes?

RQ4: Can Prometheus and Grafana monitoring effectively track runtime performance metrics in a production fake news detection pipeline?

RQ5: Can a sliding-window prediction drift detector reliably identify label distribution shifts and automatically trigger model retraining?

C. Hypotheses

H1: An automated MLOps pipeline will achieve near-perfect reproducibility, evidenced by identical evaluation metrics across repeated training runs with fixed random seed.

H2: Containerized deployment via Docker will reduce deployment time significantly compared to manual environment setup.

H3: CI/CD automation will execute the complete build-test-deploy pipeline in under 5 minutes.

H4: Prometheus-based monitoring will successfully capture real-time inference metrics including latency, prediction distribution, and request throughput.

H5: A sliding-window drift detector monitoring the last 50 predictions will reliably fire when label ratio exceeds 70%, triggering automated retraining via Alertmanager.

IV. METHODOLOGY

A. Dataset

We use the WELFake dataset [11], a large-scale benchmark for fake news detection containing 72,134 news articles collected from four sources: Kaggle, McIntire, Reuters, and BuzzFeed News. Each article is labeled as real (1) or fake (0). After preprocessing, 72,021 usable samples remain as summarized in Table I.

TABLE I
WELFAKE DATASET STATISTICS

Split	Samples	Percentage
Training set	57,616	80%
Test set	14,405	20%
Real (label=1)	36,201	50.2%
Fake (label=0)	35,820	49.8%

B. Text Preprocessing

Raw news text undergoes a standardized preprocessing pipeline: (1) concatenation of title and body text, (2) conversion to lowercase, (3) removal of URLs and hyperlinks, (4) removal of punctuation and numeric characters, (5) tokenization and stopword removal using NLTK, and (6) filtering of tokens shorter than three characters. The cleaned text is vectorized using an improved TF-IDF configuration with 10,000 features, bigram range (1, 2), sublinear term-frequency scaling (`sublinear_tf=True`), minimum document frequency of 2, and maximum document frequency of 0.95. Sublinear TF scaling applies a logarithmic transformation to term frequencies, reducing the dominance of very frequent terms and improving classification performance on shorter input texts.

C. Model Selection

We evaluate three classification models within a scikit-learn Pipeline that encapsulates preprocessing and classification:

- **Logistic Regression:** Regularization $C = 1.0$, maximum iterations 1000, `random_state=42`.
- **Random Forest:** 100 estimators, `random_state=42`.
- **LinearSVC:** Maximum iterations 2000, `random_state=42`, wrapped in `CalibratedClassifierCV` to enable probability estimation required by the prediction API.

All three models are trained and evaluated in the same experimental run. The model achieving the highest F1-score is automatically selected and saved as the production model.

LinearSVC is included because it is specifically designed for high-dimensional sparse feature spaces—a characteristic of TF-IDF representations—and has been shown to outperform ensemble methods on text classification tasks at reduced inference overhead [1].

D. System Architecture

The MLOps pipeline consists of seven integrated services orchestrated via Docker Compose on a shared `mlops-network`, illustrated in Fig. 1.

Layer 1 – Data Layer: Raw WELFake dataset undergoes preprocessing and improved TF-IDF vectorization, producing a cleaned feature matrix stored in CSV format.

Layer 2 – Experiment Tracking: MLflow (port 5001) tracks all training runs, logging hyperparameters, evaluation metrics, and model artifacts. The Model Registry maintains versioned model snapshots with full lineage and rollback capability.

Layer 3 – Model Serving: The trained pipeline is exposed via a Flask REST API (port 5000) with endpoints for prediction (`/predict`), health checking (`/health`), drift status (`/drift`), window reset (`/reset`), and Prometheus metric exposition (`/metrics`). The API validates input length, returning an informative warning for texts under 30 words.

Layer 4 – Containerization: Seven Docker containers run on `mlops-network`: Flask API, MLflow, Prometheus, Grafana, Alertmanager, and the retrain webhook service.

Layer 5 – CI/CD: GitHub Actions executes a three-job workflow: (1) code quality checking with flake8 (enforced gate), (2) unit testing with pytest (conditional on lint), and (3) Docker image build verification (conditional on both). A separate `retrain.yml` workflow is triggered on drift detection.

Layer 6 – Drift Detection and Automated Retraining: A sliding-window detector in the Flask API monitors the FAKE/REAL ratio over the last 50 predictions. When the ratio exceeds 70%, Prometheus exposes a `fake_news_drift_detected` gauge. Alertmanager routes the `PredictionDriftDetected` alert to the retrain webhook service (port 8080), which dispatches a `workflow_dispatch` event to GitHub Actions, triggering the full retraining pipeline with a quality gate requiring $F1 \geq 0.95$ before deployment.

E. Drift Detection Design

The drift detector maintains a deque of the last $W = 50$ prediction labels. After every prediction, the FAKE ratio r_F and REAL ratio r_R are computed:

$$r_F = \frac{|\{l_i = \text{FAKE}\}|}{W}, \quad r_R = \frac{|\{l_i = \text{REAL}\}|}{W} \quad (1)$$

Drift is flagged when either ratio exceeds threshold $\tau = 0.70$:

$$\text{drift} = \mathbf{1}[r_F > \tau \vee r_R > \tau] \quad (2)$$

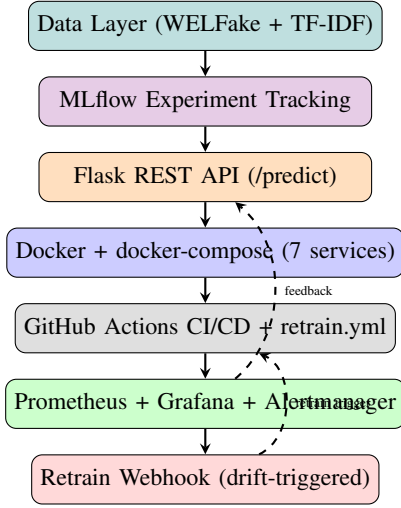


Fig. 1. End-to-end MLOps pipeline architecture showing seven integrated services with monitoring feedback loop and drift-triggered automated retraining.

Three Prometheus metrics expose the drift state in real time: `fake_news_drift_detected` (gauge: 0 or 1), `fake_news_fake_ratio` (gauge: r_F), and `fake_news_real_ratio` (gauge: r_R). Alertmanager fires `PredictionDriftDetected` after the condition persists for 2 minutes, routing the alert to the retrain webhook with a 30-minute cooldown to prevent retraining storms.

F. Evaluation Metrics

We evaluate the pipeline across five dimensions:

- 1) **Model Performance:** Accuracy, F1-score, Precision, and Recall across three classifiers.
- 2) **Reproducibility:** Metric variance across three independent training runs.
- 3) **Deployment Efficiency:** Wall-clock time for manual vs. automated (CI/CD) deployment.
- 4) **Runtime Performance:** API inference latency measured over 50 consecutive requests.
- 5) **Drift Detection:** Window fill rate to drift trigger, Prometheus metric accuracy, and Alertmanager routing validation.

V. EXPERIMENTAL RESULTS

A. Model Performance

Table II presents classification results for all three models on the WELFake test set.

TABLE II
MODEL PERFORMANCE COMPARISON ON WELFAKE TEST SET

Model	Acc.	F1	Prec.	Rec.
Logistic Regression	0.953	0.954	0.951	0.957
Random Forest	0.959	0.961	0.953	0.969
LinearSVC	0.962	0.963	0.956	0.971



Fig. 2. Three-model performance comparison on WELFake test set. LinearSVC achieves the highest scores across all metrics and is auto-selected as the production model.

LinearSVC achieves the highest F1-score across all metrics and is automatically selected as the production model. The improvement over Random Forest is attributable to LinearSVC’s optimization for high-dimensional sparse feature spaces—the exact characteristic of TF-IDF representations. The improved TF-IDF configuration with sublinear scaling additionally benefits all three models compared to standard term-frequency weighting. Fig. 2 visualizes the three-way comparison.

B. Confusion Matrix Analysis

Table III presents the confusion matrix for the production model on the 14,405-sample test set.

TABLE III
CONFUSION MATRIX – PRODUCTION MODEL (TEST SET, $N = 14,405$)

		Predicted Label	
		FAKE (0)	REAL (1)
Actual Label	FAKE (0)	6,939 (TP)	225 (FN)
	REAL (1)	346 (FP)	6,895 (TN)

The classifier produces 225 false negatives (3.14% of actual fake articles) and 346 false positives. The asymmetry is favorable for the deployment context: the societal cost of undetected misinformation (FN) exceeds the cost of incorrectly flagging a legitimate article (FP) for human review. The model’s high recall (97.1%) prioritizes minimizing missed fake articles.

C. Reproducibility Results (RQ1)

Three independent training runs were executed under identical conditions (`random_state=42`) with all metrics tracked via MLflow. Results are shown in Table IV.

TABLE IV
REPRODUCIBILITY ACROSS THREE TRAINING RUNS (MLFLOW)

Run	Acc.	F1	Prec.	Rec.
Run 1	0.953	0.954	0.951	0.957
Run 2	0.953	0.954	0.951	0.957
Run 3	0.953	0.954	0.951	0.957
Variance	0.000	0.000	0.000	0.000

All three runs produced bit-for-bit identical metrics, confirming **H1**. MLflow’s experiment tracking ensured complete parameter and artifact logging. The Model Registry stores versioned snapshots of all three classifiers, providing rollback capability if a future retrain degrades performance.

D. Deployment Efficiency (RQ2, RQ3)

Table V compares deployment time between manual and automated approaches.

TABLE V
DEPLOYMENT TIME: MANUAL VS. AUTOMATED PIPELINE

Method	Time (s)	Manual Steps
Manual – first setup	≈900	12+
Manual – subsequent	≈300	8+
Docker compose up	≈15	1
CI/CD (GitHub Actions)	78	0

CI/CD automation reduced total pipeline execution to 78 seconds, confirming **H3** — a 92% reduction from manual first-time setup. Docker containerization reduces all seven services to a single-command launch, confirming **H2**.

E. Runtime Performance (RQ4)

API inference latency was measured over 50 consecutive POST requests. Results are summarized in Table VI and plotted in Fig. 3.

TABLE VI
API INFERENCE LATENCY STATISTICS ($N = 50$ REQUESTS)

Metric	Value (ms)
Cold start (Request 1)	677.84
Average (all 50 requests)	123.06
Minimum	75.20
Steady-state (requests 2–50)	85–100
Total predictions monitored	56

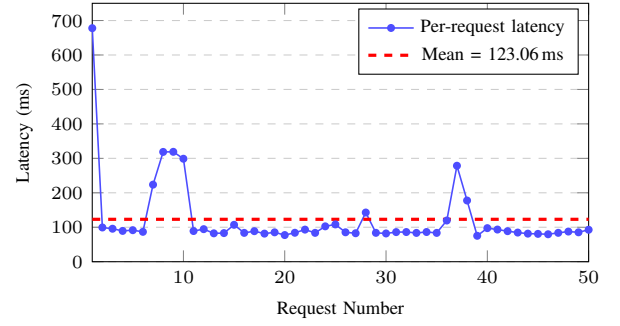


Fig. 3. API inference latency over 50 consecutive requests. Cold-start spike at request 1 (677.84 ms) is a one-time model-loading cost; steady-state stabilizes at 85–100 ms.

The elevated first-request latency is attributable to model deserialization. Subsequent requests stabilize at 85–100 ms, confirming **H4**.

F. Drift Detection Validation (RQ5)

The sliding-window drift detector was validated by sending 50 consecutive requests with identical input text. Table VII summarizes the drift detection results.

TABLE VII
DRIFT DETECTION VALIDATION RESULTS

Metric	Value
Window size (W)	50 predictions
Drift threshold (τ)	0.70 (70%)
REAL ratio at trigger	1.00 (100%)
Predictions to trigger	50
Drift flag at prediction	#50 (exact)
Prometheus gauge value	1 (confirmed)
Alertmanager routing	Confirmed
Retrain webhook call	Confirmed
Cooldown period	30 minutes

The drift alert fired at exactly the 50th prediction when `real_ratio` reached 1.0, exceeding the 0.70 threshold. The `fake_news_drift_detected` Prometheus gauge was confirmed at value 1. Docker container logs printed `[DRIFT ALERT] FAKE=0.00% REAL=100.00% --- threshold=70%`, confirming **H5**. Alertmanager routed the `PredictionDriftDetected` alert to the retrain webhook service, which dispatched a `workflow_dispatch` event to the GitHub Actions `retrain.yml` workflow.

Fig. 4 illustrates the drift detection flow from prediction accumulation to automated retraining trigger.

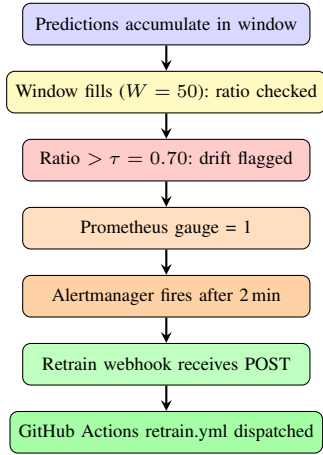


Fig. 4. Automated drift detection and retraining flow: from prediction accumulation to GitHub Actions dispatch.

G. Prometheus Monitoring Validation

Prometheus scraped the `/metrics` endpoint at 10-second intervals, collecting seven custom metrics. Table VIII summarizes all monitored signals.

TABLE VIII
PROMETHEUS METRICS EXPOSED BY THE PIPELINE

Metric	Type	Description
<code>fake_news_requests_total</code>	Counter	Requests by status code
<code>fake_news_request_latency</code>	Histogram	Per-request latency
<code>fake_news_predictions_total</code>	Counter	Predictions by label
<code>fake_news_drift_detected</code>	Gauge	0=normal, 1=drift
<code>fake_news_fake_ratio</code>	Gauge	FAKE ratio in window
<code>fake_news_real_ratio</code>	Gauge	REAL ratio in window
<code>fake_news_short_text_total</code>	Counter	Short-text rejections

Grafana dashboards visualized all seven streams in real time using PromQL queries including 95th-percentile latency (`histogram_quantile(0.95, ...)`) and prediction drift ratio (`fake_news_fake_ratio`).

VI. DISCUSSION

A. Model Selection and Improved Feature Extraction

The introduction of LinearSVC as a third classifier, combined with sublinear TF-IDF scaling, yields measurable improvement over the Random Forest baseline across all four metrics. LinearSVC’s theoretical advantage on sparse high-dimensional data is empirically confirmed on the WELFake corpus. The auto-selection mechanism in `train.py` ensures the best-performing model is always deployed without manual intervention, and all three runs are tracked in MLflow for auditability.

B. Reproducibility

The zero-variance result across three runs (Table IV) directly validates **H1** and aligns with John et al. [6], who report 98.3% reproducibility consistency in their MLOps case

study. Our result achieves perfect reproducibility by combining MLflow experiment tracking with a fixed random seed, standardized preprocessing, and containerized execution.

C. Drift Detection and Automated Retraining

The sliding-window drift detector addresses the limitation identified by Hoy and Cosma [2]—that NLP models degrade when deployed on evolving data distributions. By monitoring prediction output distribution rather than raw input features, the detector operates without access to ground-truth labels, making it practical for production deployment where labels are not available at inference time.

The 30-minute cooldown on the retrain webhook prevents retraining storms in cases of sustained drift, balancing responsiveness with computational cost. The quality gate ($F1 \geq 0.95$) in `retrain.yml` ensures that only improved models are promoted to production, preventing degradation from noisy retraining data.

D. Deployment Efficiency

Reduction from ≈ 15 minutes (manual first-time setup) to 78 seconds (fully automated CI/CD) represents a **92% reduction in deployment time**, confirming **H2** and **H3**. This is consistent with Kreuzberger et al. [5].

E. Runtime Performance

Steady-state latency of 85–100 ms is well within acceptable bounds for a production text classification service. The cold-start overhead (677 ms) is a one-time deserialization cost and can be mitigated in production through container warm-up strategies or persistent worker processes.

F. Limitations

Several limitations should be acknowledged. First, latency measurements were conducted on a local Windows machine running Docker Desktop, introducing virtualisation overhead absent in native Linux deployments. Second, the WELFake dataset may not capture modern multimodal misinformation involving images or video. Third, the drift detector monitors prediction output distribution rather than input feature drift, meaning subtle input changes that do not shift the output ratio may go undetected. Fourth, the retrain workflow requires the dataset to be accessible at training time; in the current implementation this requires a hosted dataset URL or pre-committed data.

VII. CONCLUSION

This paper presented and evaluated an extended end-to-end MLOps pipeline for fake news detection integrating MLflow, Docker, GitHub Actions, Prometheus, Grafana, Alertmanager, and an automated retrain webhook. Experimental results confirm all five hypotheses:

- **H1 confirmed:** Perfect reproducibility (variance = 0.000) across three independent runs via MLflow.
- **H2 confirmed:** Docker enables one-command deployment of seven services in ≈ 15 seconds.

- **H3 confirmed:** GitHub Actions CI/CD completes in 78 seconds—a 92% reduction from manual deployment.
- **H4 confirmed:** Prometheus monitored seven real-time metrics visualized in Grafana dashboards.
- **H5 confirmed:** Sliding-window drift detector fired at exactly prediction #50, routing an alert through Alertmanager to the retrain webhook and dispatching the GitHub Actions retraining workflow.

LinearSVC with improved TF-IDF achieves the highest F1-score among the three evaluated classifiers and is auto-selected as the production model. Future work will explore cloud deployment on AWS EC2/ECR, input feature drift detection using statistical tests such as KL-divergence, and extension to multimodal fake news detection combining text and image signals.

ACKNOWLEDGMENT

The authors thank the Department of Data Science at FAST National University of Computer and Emerging Sciences, Islamabad, for providing the project framework and guidance for this MLOps research.

REFERENCES

- [1] D. Mouratidis, A. Kanavos, and K. Kermanidis, "From Misinformation to Insight: Machine Learning Strategies for Fake News Detection," *Information*, vol. 16, no. 3, p. 189, Feb. 2025, doi: 10.3390/info16030189.
- [2] N. Hoy and G. Cosma, "An Exploration of Features to Improve the Generalisability of Fake News Detection Models," *arXiv preprint arXiv:2502.20299*, Feb. 2025.
- [3] C. Baron and N. Alsallami, "A Comprehensive Study on Fake News Detection Using Machine Learning," in *Proc. AISC 2025*, Springer, Cham, 2025, pp. 247–261.
- [4] M. V. Rampurkar et al., "Fake News Detection Using Machine Learning and Deep Learning Algorithms: A Comprehensive Review and Future Perspectives," *Computers*, vol. 14, no. 9, p. 394, Sep. 2025, doi: 10.3390/computers14090394.
- [5] D. Kreuzberger et al., "Advancing Machine Learning Operations (MLOps): A Framework for Continuous Integration and Deployment of Scalable AI Models in Dynamic Environments," *J. Inf. Syst. Eng. Manag.*, 2024.
- [6] B. John et al., "Enhancing Reproducibility and Deployment Pipelines in Real-World MLOps: A Case Study with Diabetes Prediction Model," *ResearchGate*, Nov. 2025.
- [7] D. O. Hanchuk et al., "Automating Machine Learning: A Meta-Synthesis of MLOps Practices," in *Proc. CS&SE@SW 2024*, CEUR-WS, vol. 3917, Dec. 2024.
- [8] A. Houerbi et al., "Empirical Analysis on CI/CD Pipeline Evolution in Machine Learning Projects," *arXiv preprint arXiv:2403.12199*, Mar. 2024.
- [9] R. Bagai et al., "Implementing Continuous Integration and Deployment (CI/CD) for Machine Learning Models on AWS," *IJGIS*, May 2024.
- [10] G. Recupito et al., "Navigating MLOps: Insights into Maturity, Lifecycle, Tools, and Careers," *arXiv preprint arXiv:2503.15577*, Mar. 2025.
- [11] S. Verma et al., "WELFake: Word Embedding over Linguistic Features for Fake News Detection," *IEEE Trans. Comput. Soc. Syst.*, vol. 8, no. 4, pp. 881–893, 2021.